# Analysis of Cache Performance

## 1. Fixed L1d Cache (1kB) – Varying L1i Cache Size

| Benchmark | 16B | 128B | 512B | 1024B |
|---|---|---|---|---|
| descending | 0.022976 | 0.114890 | 0.100362 | 0.093962 |
| evenodd | 0.022140 | 0.021739 | 0.021352 | 0.020979 |
| fibonacci | 0.020516 | 0.049087 | 0.047101 | 0.045270 |
| l1d benchmark | 0.019212 | 0.018957 | 0.018709 | 0.018467 |
| l1i benchmark | 0.020531 | 0.020384 | 0.020238 | 0.020095 |
| palindrome | 0.022962 | 0.066038 | 0.062025 | 0.058473 |
| prime | 0.023052 | 0.044892 | 0.043027 | 0.041311 |

Table 1: IPC vs L1i Cache Size (L1d Fixed at 1kB)

IPC vs L1i Cache Size

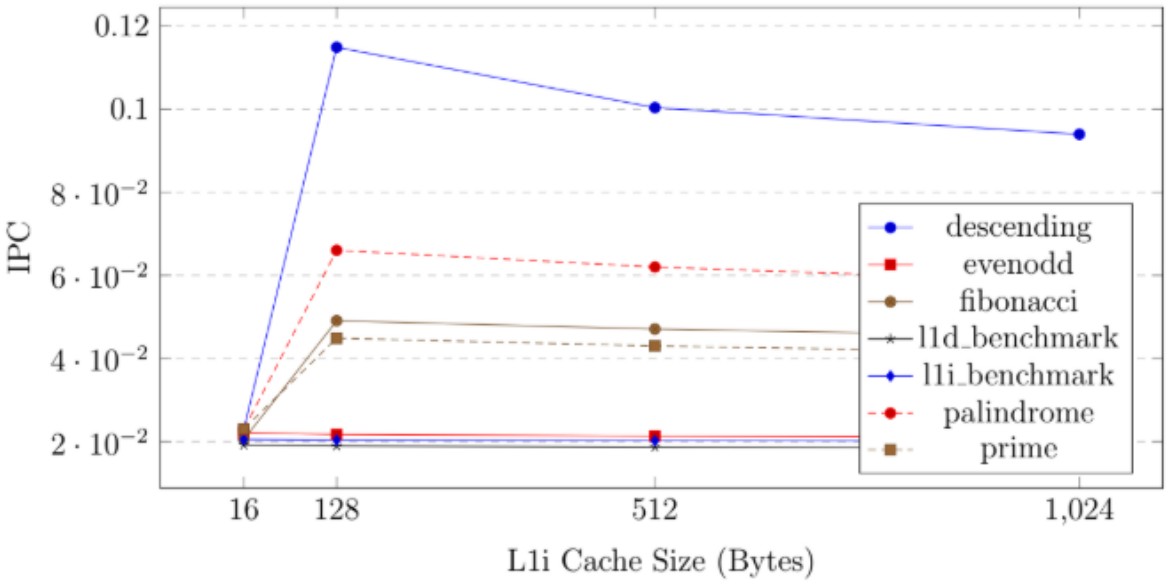



Figure 1: IPC for Varying L1i Cache Size (L1d fixed at 1kB)

2. Fixed L1i Cache (1kB) – Varying L1d Cache Size

| Benchmark | 16B | 128B | 512B | 1024B |
|---|---|---|---|---|
| descending | 0.071817 | 0.100654 | 0.097984 | 0.093962 |
| evenodd | 0.021201 | 0.021127 | 0.021053 | 0.020979 |
| fibonacci | 0.048089 | 0.046763 | 0.046318 | 0.045270 |
| l1d benchmark | 0.019139 | 0.018886 | 0.018744 | 0.018467 |
| l1i benchmark | 0.021767 | 0.021092 | 0.020706 | 0.020095 |
| palindrome | 0.058683 | 0.058612 | 0.058542 | 0.058473 |
| prime | 0.041488 | 0.041429 | 0.041369 | 0.041311 |

Table 2: IPC vs L1d Cache Size (L1i Fixed at 1kB)

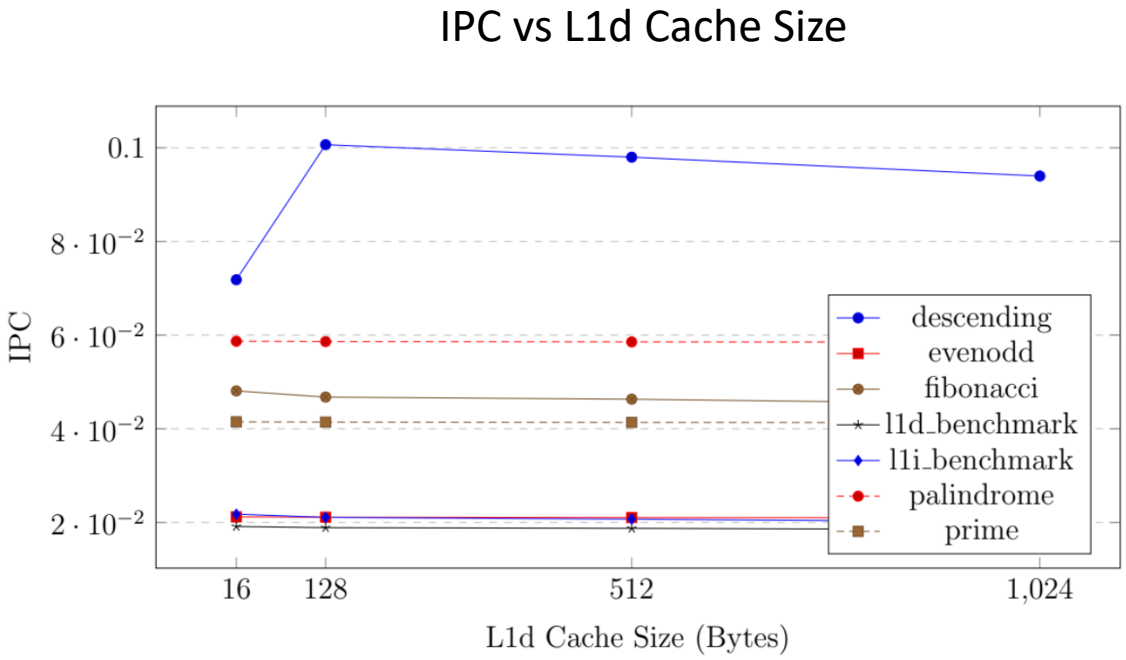



Figure 2: IPC for Varying L1d Cache Size (L1i fixed at 1kB)

# Benchmark Program: l1d benchmark.asm

```
.data
arr:
1
2
3
4
5
6
7
8
9
10
dummy:
0
0
0
0
0
0
0
0
0
0
.text
main:
load %x0, $arr, %x3
```

```
addi %x3, 1, %x3
store %x3, $arr, %x0
load %x0, $arr, %x1
addi %x1, 1, %x1
load %x1, 0, %x4
addi %x4, 1, %x4
store %x4, 0, %x1
load %x0, $arr, %x1
addi %x1, 2, %x1
load %x1, 0, %x5
addi %x5, 1, %x5
store %x5, 0, %x1
load %x0, $arr, %x1
addi %x1, 3, %x1
load %x1, 0, %x6
addi %x6, 1, %x6
store %x6, 0, %x1
end
```

# Benchmark Program: l1i benchmark.asm

```
.data
a:
0
b:
1000
c:
2000
d:
```

```
3000
e:
4000
.text
main:
load %x0, $a, %x3
addi %x3, 1, %x3
store %x3, $a, %x0
load %x0, $b, %x4
addi %x4, 1, %x4
store %x4, $b, %x0
load %x0, $c, %x5
addi %x5, 1, %x5
store %x5, $c, %x0
load %x0, $d, %x6
addi %x6, 1, %x6
store %x6, $d, %x0
load %x0, $e, %x7
addi %x7, 1, %x7
store %x7, $e, %x0
end
```

## Final Observations and Correlation

- **Descending:** Exhibits high instruction and data reuse; IPC improves with enhancements in both L1 instruction (L1i) and L1 data (L1d) caches.
- **Even-Odd:** Characterized by a minimal static instruction footprint; performance remains largely unaffected by cache size variations.
- **Fibonacci:** Shows greater sensitivity to the L1i cache due to repeated execution of looped instructions.
- **L1d Benchmark:** Involves large, scattered data access patterns; IPC improves significantly with increased L1d cache capacity.
- **L1i Benchmark:** Features a highly repetitive instruction stream; performance is strongly dependent on L1i cache size.
- **Palindrome:** Contains balanced, branch-intensive logic; IPC benefits from larger L1i cache configurations.
- **Prime:** Demonstrates moderate control and data flow complexity; IPC scales proportionally with improvements in both instruction and data caches